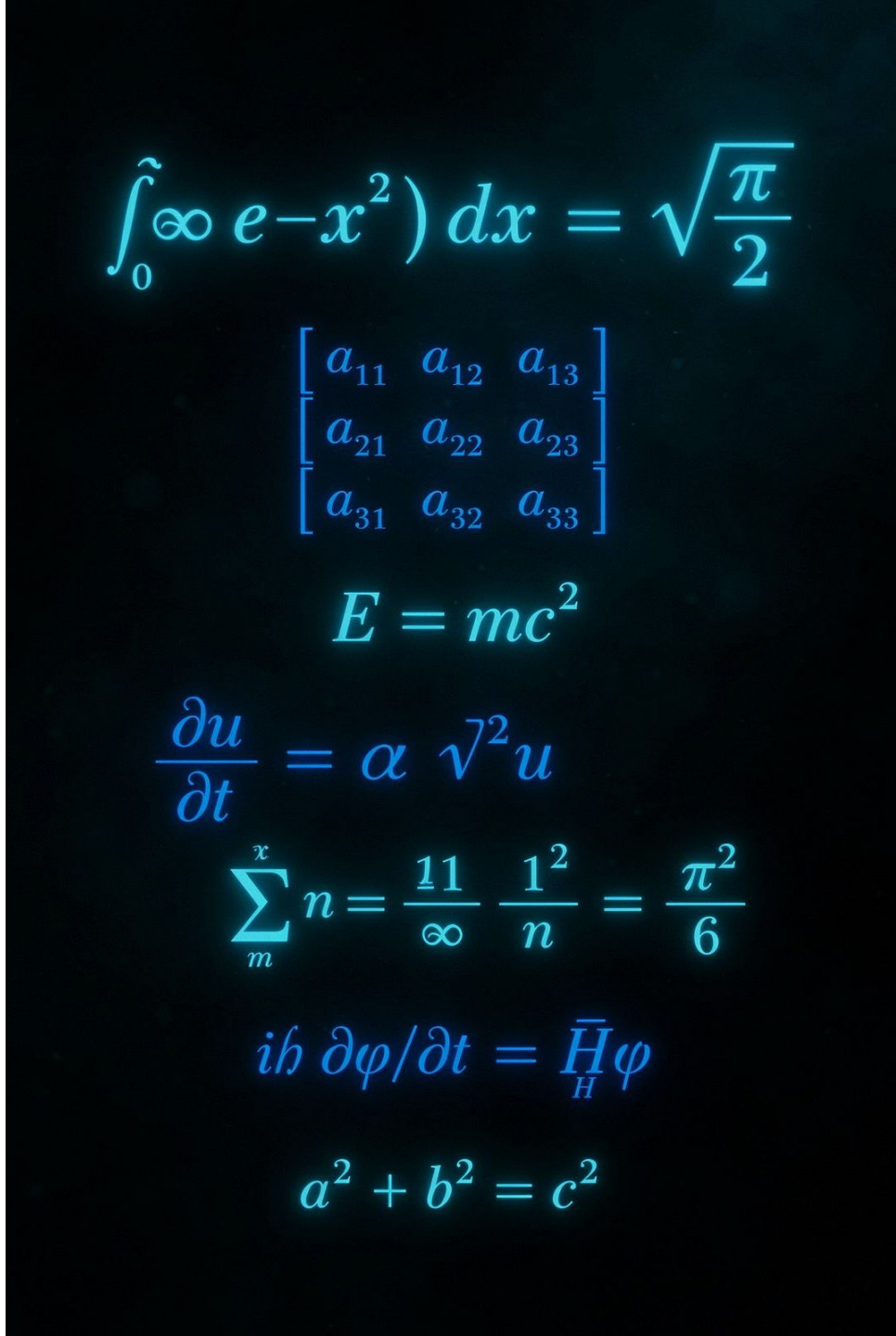
Sprint 5 · Matemáticas y estadística en código

S5.1 · NumPy y álgebra lineal

Domina las herramientas numéricas esenciales de Python: desde arrays eficientes hasta operaciones de álgebra lineal aplicada. Este sprint te prepara para trabajar con datos a escala real.



Python puro vs. NumPy

❌ Lista de Python

```
data = list(range(1_000_000))
total = 0
for x in data:
    total += x * 2
```

Cada elemento es un objeto Python completo. El bucle interpreta instrucción por instrucción. Lento y costoso en memoria.

✅ Array de NumPy

```
import numpy as np
data = np.arange(1_000_000)
total = (data * 2).sum()
```

Los datos se almacenan como valores contiguos en memoria. Las operaciones se ejecutan en C vectorizado. Hasta **100× más rápido**.

NumPy es la piedra angular de pandas, scikit-learn, TensorFlow y prácticamente todo el ecosistema científico de Python.

Creación de arrays: recetario

```
import numpy as np

# Desde una lista
a = np.array([1, 2, 3, 4])        # dtype=int64 por defecto

# Rangos
b = np.arange(0, 10, 2)           # [0, 2, 4, 6, 8]
c = np.linspace(0, 1, 5)          # [0.0, 0.25, 0.5, 0.75, 1.0]

# Arrays especiales
zeros  = np.zeros((3, 4))         # Matriz 3×4 de ceros
ones   = np.ones((2, 2), dtype=np.float32)
eye    = np.eye(3)                # Identidad 3×3
rand   = np.random.rand(3, 3)     # Valores aleatorios [0,1)

# Atributos clave
print(a.shape, a.dtype, a.ndim, a.size)
```

Conocer los atributos `shape`, `dtype`, `ndim` y `size` es imprescindible para depurar errores de dimensiones en modelos y pipelines de datos.

Tipos de indexación

Indexación básica y slicing

```
a = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

a[0, 1]     # 2  (fila 0, col 1)
a[1:, :2]   # [[4,5],[7,8]]
a[:, 2]     # [3, 6, 9] (col entera)
```

La sintaxis `[inicio:fin:paso]` funciona en cada dimensión de forma independiente.

Indexación booleana y fancy

```
mask = a > 5
a[mask]         # [6, 7, 8, 9]

# Fancy indexing
idx = np.array([0, 2])
a[idx, :]       # filas 0 y 2

# Modificar con máscara
a[a % 2 == 0] = 0  # pares → 0
```

Las máscaras booleanas son la forma idiomática de filtrar filas en DataFrames y arrays sin bucles.

Vectorización en la práctica

```
import numpy as np
import time

n = 10_000_000
a = np.random.rand(n)

#
```

Reglas del broadcasting



Ejemplo práctico

```
a = np.array([[1],[2],[3]])  # (3,1)
b = np.array([10, 20, 30]) # (3,)
# → broadcast a (3,3)
a + b
# [[11,21,31],
#  [12,22,32],
#  [13,23,33]]
```

Normalización de datos

```
X = np.random.rand(100, 5)
# Restar media de cada columna
X_norm = X - X.mean(axis=0)
# Dividir por desviación típica
X_std  = X_norm / X.std(axis=0)
```

Broadcasting hace que la normalización de matrices sea una sola línea, sin bucles sobre columnas.

Producto escalar y multiplicación matricial

np.dot — producto escalar

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.dot(a, b)   # 1·4+2·5+3·6 = 32

# También funciona con matrices
A = np.random.rand(3, 4)
B = np.random.rand(4, 2)
C = np.dot(A, B)  # (3,2)
```

@ — operador matmul

```
A = np.random.rand(3, 4)
B = np.random.rand(4, 2)

# Equivalente moderno y legible
C = A @ B       # (3,2)

# np.matmul vs np.dot:
# matmul NO acepta escalares
# dot SÍ acepta escalares y 1-D
# Prefiere @ para matrices
```

El operador `@` (PEP 465) es la forma recomendada en código moderno.

Inversa, determinante y sistemas lineales

```
from numpy import linalg as la

A = np.array([[2., 1.],
              [5., 3.]])

# Inversa
A_inv = la.inv(A)          # A @ A_inv ≈ I

# Determinante
det = la.det(A)            # 1.0

# Resolver sistema Ax = b  (más eficiente que inv)
b = np.array([1., 2.])
x = la.solve(A, b)         # [1., -1.]

# Verificar
np.allclose(A @ x, b)      # True
```

⚠ Usar `la.solve` en lugar de `la.inv(A) @ b` es más estable numéricamente y entre 2-3× más rápido para matrices grandes.

Generadores y distribuciones

```
import numpy as np

# API moderna (recomendada desde NumPy 1.17)
rng = np.random.default_rng(seed=42)

# Distribuciones
rng.random((3, 3))            # Uniforme [0, 1)
rng.integers(0, 10, size=5)   # Enteros aleatorios
rng.normal(loc=0, scale=1, size=1000)  # Gaussiana N(0,1)
rng.choice([1, 2, 3, 4], size=10, replace=True)

# Barajar y muestrear
data = np.arange(20)
rng.shuffle(data)             # In-place
sample = rng.choice(data, size=5, replace=False)

# API antigua (compatible pero menos recomendada)
np.random.seed(42)
np.random.randn(3, 3)
```

Fija siempre una semilla en tus notebooks para que los resultados sean **reproducibles** entre ejecuciones y entre compañeros de equipo.

S05 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S05_01_numpy — 35 min

Arrays, vectorización y *broadcasting*

sprints/S05/notebooks/S05_01_numpy.ipynb

CUADERNO

S05_02_algebra_lineal — 35 min

Matrices, sistemas y ecuación normal

sprints/S05/notebooks/S05_02_algebra_lineal.ipynb